

练习

对实验报告的要求：

- 基于markdown格式来完成，以文本方式为主
- 填写各个基本练习中要求完成的报告内容
- 列出你认为本实验中重要的知识点，以及与对应的OS原理中的知识点，并简要说明你对二者的含义，关系，差异等方面的理解（也可能出现实验中的知识点没有对应的原理知识点）
- 列出你认为OS原理中很重要，但在实验中没有对应上的知识点

练习0：填写已有实验

本实验依赖实验2/3/4。请把你做的实验2/3/4的代码填入本实验中代码中有“LAB2”/“LAB3”/“LAB4”的注释相应部分。注意：为了能够正确执行lab5的测试应用程序，可能需对已完成的实验2/3/4的代码进行进一步改进。

```
proc->state = PROC_UNINIT;  
proc->pid = -1;  
  
proc->state = 0;  
proc->cptr = NULL; // 初始化 cptr 为 NULL，表示没有子进程  
proc->yptr = NULL; // 初始化 yptr 为 NULL，表示没有“年轻”兄弟进程  
proc->optr = NULL; // 初始化 optr 为 NULL，表示没有“老”兄弟进程  
  
proc->runs = 0;  
proc->kstack = 0;  
proc->need_resched = 0;  
proc->parent = NULL;  
proc->mm = NULL;  
memset(&(proc->context), 0, sizeof(struct context));  
proc->tf = NULL;  
proc->cr3 = boot_cr3;  
proc->flags = 0;  
memset(proc->name, 0, PROC_NAME_LEN + 1);
```

```
// 1. call alloc_proc to allocate a proc_struct  
// 调用alloc_proc，首先获得一块用户信息块。  
if ((proc = alloc_proc()) == NULL)  
{  
    goto fork_out;  
}  
  
// 更新1：当前进程的wait_state是0  
current->wait_state = 0;  
  
// 2. call setup_kstack to allocate a kernel stack for child process  
// 为进程分配一个内核栈。  
proc->parent = current;  
if (setup_kstack(proc))
```

```

{
    goto bad_fork_cleanup_kstack;
}
// 3. call copy_mm to dup OR share mm according clone_flag
// 复制原进程的内存管理信息到新进程（但内核线程不必做此事）
if (copy_mm(clone_flags, proc))
{
    goto bad_fork_cleanup_proc;
}
// 4. call copy_thread to setup tf & context in proc_struct
// 复制原进程上下文到新进程
copy_thread(proc, stack, tf);
// 5. insert proc_struct into hash_list && proc_list
// 将新进程添加到进程列表
bool intr_flag;
local_intr_save(intr_flag);
{
    proc->pid = get_pid();
    hash_proc(proc);
    //list_add(&proc_list, &(proc->list_link));

    // 更新2: 设置进程间的关系链接
    set_links(proc);
}
local_intr_restore(intr_flag);
// 6. call wakeup_proc to make the new child process RUNNABLE
// 唤醒新进程
wakeup_proc(proc);
// 7. set ret vaule using child proc's pid
//返回新进程号
ret = proc->pid;

```

练习1: 加载应用程序并执行（需要编码）

`do_execv`函数调用 `load_icode`（位于 `kern/process/proc.c`中）来加载并解析一个处于内存中的ELF执行文件格式的应用程序。你需要补充 `load_icode` 的第6步，建立相应的用户内存空间来放置应用程序的代码段、数据段等，且要设置好 `proc_struct` 结构中的成员变量 `trapframe` 中的内容，确保在执行此进程后，能够从应用程序设定的起始执行地址开始执行。需设置正确的 `trapframe` 内容。

请在实验报告中简要说明你的设计实现过程。

- 请简要描述这个用户态进程被 `ucore` 选择占用CPU执行（RUNNING态）到具体执行应用程序第一条指令的整个经过。

```

tf->gpr.sp = USTACKTOP;
tf->epc = elf->e_entry;
tf->status = sstatus & ~(SSTATUS_SPP | SSTATUS_SPIE);

```

1. 在 `init_main` 中通过 `kernel_thread` 调用 `do_fork` 创建并唤醒线程，使其执行函数 `user_main`，这时该线程状态已经为 `PROC_RUNNABLE`，表明该线程开始运行
2. 在 `user_main` 中通过宏 `KERNEL_EXECVE`，调用 `kernel_execve`

3. `do_execve()` `load_icode()` 里面只是构建了用户程序运行的上下文，但是并没有完成切换。上下文切换实际上要借助中断处理的返回来完成。直接调用 `do_execve()` 是无法完成上下文切换的。（如果是在用户态调用 `exec()`，系统调用的 `ecall` 产生的中断返回时，就可以完成上下文切换）由于目前我们在 S mode 下，所以不能通过 `ecall` 来产生中断，用 `ebreak` 产生断点中断进行处理，通过设置 `a7` 寄存器的值为 10 说明这不是一个普通的断点中断，而是要转发到 `syscall()`，这样用一个不是特别优雅的方式，实现了在内核态使用系统调用。
4. 在 `kernel_execve` 中执行 `ebreak`，发生断点异常，转到 `__alltraps`，转到 `trap`，再到 `trap_dispatch`，然后到 `exception_handler`，最后到 `CAUSE_BREAKPOINT` 处
5. 在 `CAUSE_BREAKPOINT` 处调用 `syscall`
6. 在 `syscall` 中根据参数，确定执行 `sys_exec`，调用 `do_execve`
7. 在 `do_execve` 中调用 `load_icode`，加载文件
8. 创建一个新的 `mm_struct`。
9. 创建一个新的 PDT，将 `mm` 的 `pgdir` 设置为这个 PDT 的虚拟地址。
10. 读取 ELF 格式，检验其合法性，循环读取每一个程序段，将需要加载的段加载到内存中，设置相应段的权限。之后初始化 BSS 段，将其清零。
11. 设置用户栈。
12. 设置当前进程的 `mm`，`cr3`，设置 `satp` 寄存器。
13. 设置 `trapframe`，将 `gpr.sp` 指向用户栈顶，将 `epc` 设置为 ELF 文件的入口地址，设置 `sstatus` 寄存器，将 `SSTATUS_SPP` 位置 0，表示退出当前中断后进入用户态，将 `SSTATUS_SPIE` 位置 1，表示退出当前中断后开启中断。

14.

```

        .globl __alltraps
__alltraps:
    SAVE_ALL

    move a0, sp
    jal trap
    # sp should be the same as before "jal trap"

    .globl __trapret
__trapret:
    RESTORE_ALL
    # return from supervisor call
    sret

```

加载完毕后一路返回，直到 `__alltraps` 的末尾，接着执行 `__trapret` 后的内容，到 `sret`，表示退出 S 态，回到用户态执行，这时开始执行用户的应用程序

练习2: 父进程复制自己的内存空间给子进程（需要编码）

创建子进程的函数 `do_fork` 在执行中将拷贝当前进程（即父进程）的用户内存地址空间中的合法内容到新进程中（子进程），完成内存资源的复制。具体是通过 `copy_range` 函数（位于 `kern/mm/pmm.c` 中）实现的，请补充 `copy_range` 的实现，确保能够正确执行。

请在实验报告中简要说明你的设计实现过程。

- 如何设计实现 Copy on Write 机制？给出概要设计，鼓励给出详细设计。

Copy-on-write (简称COW) 的基本概念是指如果有多个使用者对一个资源A (比如内存块) 进行读操作, 则每个使用者只需获得一个指向同一个资源A的指针, 就可以该资源了。若某使用者需要对这个资源A进行写操作, 系统会对该资源进行拷贝操作, 从而使得该“写操作”使用者获得一个该资源A的“私有”拷贝—资源B, 可对资源B进行写操作。该“写操作”使用者对资源B的改变对于其他的使用者而言是不可见的, 因为其他使用者看到的还是资源A。

```
uintptr_t* src = page2kva(page);
uintptr_t* dst = page2kva(npage);
memcpy(dst, src, PGSIZE);
ret = page_insert(to, npage, start, perm);
```

在 `copy_range` 中实现了将父进程的内存空间复制给子进程的功能。逐个内存页进行复制, 首先找到父进程的页表项, 然后创建一个子进程新的页表项, 设置对应的权限, 然后将父进程的页表项对应的内存页复制到子进程的页表项对应的内存页中, 然后将子进程的页表项加入到子进程的页表中。

如何设计实现 Copy on Write 机制?

要实现 Copy on Write 机制, 可以在复制父进程的内存空间给子进程时, 不复制整个内存页, 而是只复制页表项, 然后将父进程和子进程的页表项的权限都设置为只读。这样两个进程都可以访问同一个内存页, 当其中一个进程要写入时, 会触发缺页异常, 然后在缺页异常处理函数中, 复制整个内存页, 设置可写入权限, 这时就可以写入了。当某个共享页面只剩下一个进程时, 就可以将其权限设置为可写。

练习3: 阅读分析源代码, 理解进程执行 `fork/exec/wait/exit` 的实现, 以及系统调用的实现 (不需要编码)

请在实验报告中简要说明你对 `fork/exec/wait/exit` 函数的分析。并回答如下问题:

- 请分析`fork/exec/wait/exit`的执行流程。重点关注哪些操作是在用户态完成, 哪些是在内核态完成? 内核态与用户态程序是如何交错执行的? 内核态执行结果是如何返回给用户程序的?
- 请给出`ucore`中一个用户态进程的执行状态生命周期图 (包执行状态, 执行状态之间的变换关系, 以及产生变换的事件或函数调用)。(字符方式画即可)

执行: `make grade`。如果所显示的应用程序检测都输出`ok`, 则基本正确。(使用的是`qemu-1.0.1`)

1. `fork`: 通过发起系统调用执行 `do_fork` 函数。用于创建并唤醒线程, 可以通过 `sys_fork` 或者 `kernel_thread` 调用。

`sys_fork()`: 把当前的进程复制一份, 创建一个子进程, 原先的进程是父进程。接下来两个进程都会收到 `sys_fork()` 的返回值, 如果返回0说明当前位于子进程中, 返回一个非0的值 (子进程的PID) 说明当前位于父进程中。然后就可以根据返回值的不同, 在两个进程里进行不同的处理。

- 初始化一个新线程
- 为新线程分配内核栈空间
- 为新线程分配新的虚拟内存或与其他线程共享虚拟内存
- 获取原线程的上下文与中断帧, 设置当前线程的上下文与中断帧
- 将新线程插入哈希表和链表中
- 唤醒新线程
- 返回线程 `id`

2. `exec`: 通过发起系统调用执行 `do_execve` 函数。用于创建用户空间, 加载用户程序, 可以通过 `sys_exec` 调用。

`sys_exec()`：在当前的进程下，停止原先正在运行的程序，开始执行一个新程序。PID不变，但是内存空间要重新分配，执行的机器代码发生了改变。我们可以用 `fork()` 和 `exec()` 配合，在当前程序不停止的情况下，开始执行另一个程序。

- 回收当前线程的虚拟内存空间
- 为当前线程分配新的虚拟内存空间并加载应用程序

3. `wait`：通过发起系统调用执行 `do_wait` 函数。用于等待线程完成，可以通过 `sys_wait` 或者 `init_main` 调用。

`sys_exit()`：退出当前的进程。

- 查找状态为 `PROC_ZOMBIE` 的子线程；如果查询到拥有子线程的线程，则设置线程状态并切换线程；如果线程已退出，则调用 `do_exit`
- 将线程从哈希表和链表中删除
- 释放线程资源

4. `exit`：通过发起系统调用执行 `do_exit` 函数。用于退出线程，可以通过 `sys_exit`、`trap`、`do_execve`、`do_wait` 调用。具体执行内容：

`sys_wait()`：挂起当前的进程，等到特定条件满足的时候再继续执行。

- 如果当前线程的虚拟内存没有用于其他线程，则销毁该虚拟内存
- 将当前线程状态设为 `PROC_ZOMBIE`，唤醒该线程的父线程
- 调用 `schedule` 切换到其他线程

扩展练习 Challenge

1. 实现 Copy on Write (COW) 机制

给出实现源码、测试用例和设计报告（包括在cow情况下的各种状态转换（类似有限状态自动机）的说明）。

这个扩展练习涉及到本实验和上一个实验“虚拟内存管理”。在ucore操作系统中，当一个用户父进程创建自己的子进程时，父进程会把其申请的用户空间设置为只读，子进程可共享父进程占用的用户内存空间中的页面（这就是一个共享的资源）。当其中任何一个进程修改此用户内存空间中的某页面时，ucore会通过page fault异常获知该操作，并完成拷贝内存页面，使得两个进程都有各自的内存页面。这样一个进程所做的修改不会被另外一个进程可见了。请在ucore中实现这样的COW机制。

由于COW实现比较复杂，容易引入bug，请参考 <https://dirtycow.ninja/> 看看能否在ucore的COW实现中模拟这个错误和解决方案。需要有解释。

这是一个big challenge.

2. 说明该用户程序是何时被预先加载到内存中的？与我们常用操作系统的加载有何区别，原因是什么？

该用户程序在操作系统加载时一起加载到内存里。我们平时使用的程序在操作系统启动时还位于磁盘中，只有当我们需要运行该程序时才会被加载到内存里。原因是在 `Makefile` 里执行了 `ld` 命令，把执行程序的代码连接在了内核代码的末尾。